

С ДЛЯ ПИТОНISTОВ • ДЕНЬ 2

Адрес

Почему `b = a` для списков не копирует список?

Вопрос дня

```
>>> a = [1, 2, 3]
>>> b = a
>>> b.append(4)
>>> a
[1, 2, 3, 4]
>>> a is b
True
```

```
int a[3] = {1, 2, 3};
int b[3] = a;
```

error: invalid initializer

Вопрос дня

```
>>> a = [1, 2, 3]
>>> b = a
>>> b.append(4)
>>> a
[1, 2, 3, 4]
>>> a is b
True
```

```
int a[3] = {1, 2, 3};
int b[3] = a;
```

error: invalid initializer

В Python строка `b = a` выполняется, но копии списка не создаёт. В C её прямой аналог не компилируется: массив нельзя присвоить массиву.

Допустимая запись — `int *b = a;`. После неё `b[0] = 999` изменит и `a[0]`, то есть повторит поведение Python.

Сегодня разбирается, что означает эта звёздочка и почему питоновский список устроен именно так.

РАЗДЕЛ 1

Адрес

У переменной есть адрес

Вчера мы условились, что память — это длинный массив байтов, и у каждого байта есть номер. Переменная занимает несколько подряд идущих байтов; номер первого из них и есть **адрес** переменной.

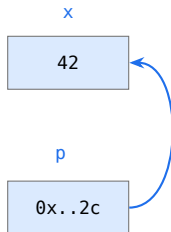
```
int x = 37;  
printf("%p\n", (void *)&x);           // 0x7ffc10a4bc2c
```



`&x` — операция «взять адрес». Печатается адрес через `%p`, и аргумент положено приводить к `void *`: `printf` не знает типов, а соврать ему мы вчера уже пробовали.

Указатель — это переменная для адреса

```
int x = 37;  
int *p = &x;    // p хранит адрес x  
  
printf("%d\n", *p);    // 37  
*p = 42;               // пишем по адресу  
printf("%d\n", x);     // 42
```



Две операции, обратные друг другу:

- `&x` — по переменной получить её адрес;
- `*p` — по адресу получить переменную. Читается «то, что лежит по адресу `p`», и годится как для чтения, так и для записи.

Звёздочка в двух ролях

```
int *p = &x;    // объявление: p имеет тип указатель" на int"  
*p = 42;        // выражение: разыменованье, запись по адресу
```

Одна и та же звёздочка означает разное в объявлении и в выражении. Отсюда соглашение писать её вплотную к имени: `int *p` читается как «`*p` имеет тип `int`».

Читайте объявления справа налево

```
int *p — p это указатель на int.  
int **pp — pp это указатель на указатель на int.  
int *a[8] — a это массив из восьми указателей на int.
```

Последняя строка понадобится в пятой задаче: такова модель питоновского списка.

Размер указателя одинаков, а тип — нет

```
sizeof(char *) == 8  
sizeof(double *) == 8  
sizeof(int **) == 8
```

Адрес — это номер байта, и он не зависит от того, что по этому номеру лежит. На нашей машине под него отведено 8 байт, поэтому все указатели одного размера.

Тогда зачем указателю тип? Затем, что по адресу надо ещё прочитать значение, а для этого нужно знать две вещи:

- **сколько байт брать** — 1 для `char`, 8 для `double`;
- **на сколько смещать** при `p + 1` — об этом через несколько слайдов.

`void *` — указатель без типа: адрес есть, читать по нему нельзя, пока не скажете, чем он является.

Указатель, который не показывает никуда

```
int *p = NULL;  
if (p != NULL) printf("%d\n", *p);    // безопасно  
printf("%d\n", *p);                  // а это UB
```

NULL — значение, про которое известно, что ни один объект по нему не лежит. Его удобно возвращать в смысле «ничего не нашлось» и удобно проверять.

Разыменование NULL

```
$ ./prog  
Segmentation fault (core dumped)  
$ gcc -fsanitize=address,undefined prog.c && ./a.out  
prog.c:5:5: runtime error: load of null pointer of type 'int'
```

Санитайзер называет строку; без него известен только сам факт падения.

Функция, меняющая переменные вызывающего

```
def swap(a, b):  
    a, b = b, a  
  
x, y = 1, 2  
swap(x, y)  
print(x, y)      # 1 2
```

```
void swap(int *a, int *b) {  
    int t = *a;  
    *a = *b;  
    *b = t;  
}  
swap(&x, &y);    // 2 1
```

Питоновская версия не работает: имена внутри функции свои, и присваивание им ничего не говорит о переменных снаружи. В Python возвращают кортеж.

Функция, меняющая переменные вызывающего

```
def swap(a, b):  
    a, b = b, a  
  
x, y = 1, 2  
swap(x, y)  
print(x, y)      # 1 2
```

```
void swap(int *a, int *b) {  
    int t = *a;  
    *a = *b;  
    *b = t;  
}  
swap(&x, &y);      // 2 1
```

Питоновская версия не работает: имена внутри функции свои, и присваивание им ничего не говорит о переменных снаружи. В Python возвращают кортеж.

Главное

В C кортежей нет и вернуть два значения нельзя. Поэтому функция получает адрес и пишет по нему. По той же причине в `scanf("%d", &n)` стоит амперсанд.

В C всё передаётся по значению

Копируется *всегда*: и число, и указатель. Разница в том, что копия указателя показывает туда же, куда оригинал.

```
void f(int *p) { *p = 5;    /* меняет то, на что показывает p */ }  
void g(int *p) { p = NULL; /* меняет только свою копию */ }
```

Как это выглядит в Python

Python ведёт себя так же, хотя это менее заметно: в функцию попадает копия ссылки на объект. Поэтому `lst.append(1)` внутри функции виден снаружи, а `lst = []` — нет.

Разница между изменяемым и неизменяемым типом в Python — это разница между изменением содержимого по указателю и заменой самого указателя, то есть между `*p = 5` и `p = NULL`.

РАЗДЕЛ 2

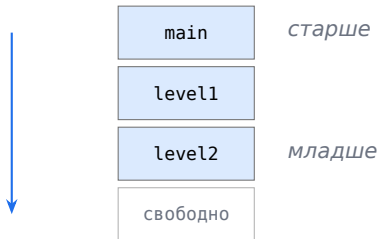
Стек

Где живут локальные переменные

```
void level2(void) { int c; }  
void level1(void) { int b; level2(); }  
int main(void)    { int a; level1(); }
```

Адреса на реальном запуске:

```
main:    &a = 0x7ffe73be3b4c  
level1:  &b = 0x7ffe73be3b2c  
level2:  &c = 0x7ffe73be3b0c
```



Каждый вызов получает свой **кадр** — участок памяти под аргументы и локальные переменные. Кадры укладываются один за другим, на нашей машине — в сторону уменьшения адресов. Отсюда и название: стек.

Кадр живёт ровно столько, сколько вызов

После `return` кадр не стирается: он перестаёт быть занятым, и следующий вызов размещает свои переменные на том же месте.

```
void save_local(int **out) { int local = 111; *out = &local; }  
void unrelated_call(void) { int other = 777; }
```

```
int *q;  
save_local(&q);           /* q получает адрес local */  
unrelated_call();        /* следующий вызов занимает тот же кадр */
```

```
save_local:    local = 111 @ 0x7fff87ee3a64  
main:         *q      = 111 @ 0x7fff87ee3a64  
unrelated_call: other = 777 @ 0x7fff87ee3a64  
main:         *q      = 777 (q не менялся)
```

Три адреса совпали: две разные функции получили одну ячейку. Значение `*q` изменилось без единого присваивания через `q`.

Указатель на мёртвый кадр

Чтение по адресу локальной переменной после выхода из функции — UB. Программа при этом обычно продолжает работать: память на месте, значение выглядит правдоподобным, пока ячейку не займёт следующий вызов.

Компилятор предупреждает об обоих способах отдать такой адрес наружу:

```
warning: function returns address of local variable [-Wreturn-local-addr]
warning: storing the address of local variable 'local' in '*out'
        [-Wdangling-pointer=]
```

Санитайзер называет происходящее тремя словами:

```
ERROR: AddressSanitizer: stack-use-after-return
```

Что компилятор делает с таким кодом

```
int *return_local(void) {  
    int local = 42;  
    return &local;      // warning: returns address of local variable  
}
```

```
return_local() вернул (nil)  
Segmentation fault (core dumped)
```

(nil) в первой строке — это `NULL`: так glibc печатает нулевой указатель по формату `%p`, вид этого вывода стандартом не задан.

Стандарт объявляет значение такого указателя неопределённым, и компилятор волен распорядиться им как угодно. gcc 15.2 подставляет `NULL` и выдаёт предупреждение; проверено на уровнях от `-O0` до `-Os`.

Другой компилятор вправе вернуть настоящий адрес, и тогда программа напечатает 42, а ошибка останется незамеченной. Полагаться нельзя ни на первое поведение, ни на второе.

Что можно возвращать

- **Значение** — всегда: оно копируется в вызывающий код.
- **Адрес того, что переживёт вызов**: глобальной переменной, строкового литерала, памяти, которую вам передали снаружи.
- **Адрес локальной переменной** — никогда.

```
int *find(int *a, size_t n, int value) {    /* так можно: массив не наш */  
    for (size_t i = 0; i < n; i++)  
        if (a[i] == value) return &a[i];  
    return NULL;                            /* не нашли */  
}
```

Пока это всё, что есть. Память, которая живёт столько, сколько нужно вам, а не сколько длится вызов, появится завтра — это куча.

Стек конечен

```
$ ulimit -s  
8192
```

Восемь мегабайт на все кадры сразу. Локальный массив на четыре миллиона `int` занимает 16 МБ и не помещается:

```
$ ./prog  
Segmentation fault (core dumped)  
  
$ gcc -fsanitize=address prog.c -o prog && ./prog  
ERROR: AddressSanitizer: stack-overflow on address 0x7ffdf4039b98
```

Главное

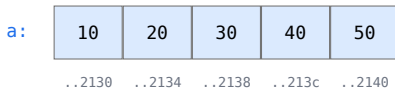
Массив на стеке хорош, пока он небольшой и его размер известен на этапе компиляции. Массив на миллион элементов, массив с размером, который стал известен во время работы, и массив, переживающий свою функцию, — это всё завтрашняя тема.

РАЗДЕЛ 3

Массивы на стеке

Массив — это N объектов подряд

```
int a[5] = {10, 20, 30, 40, 50};  
printf("%zu %zu\n", sizeof a, sizeof a / sizeof a[0]); // 20 5
```



Никакого заголовка, никакой длины, никаких границ — пять чисел, лежащих вплотную. Всё, что знает об этом массиве компилятор, он знает из объявления, которое видит прямо сейчас.

Отсюда идиома `sizeof a / sizeof a[0]`: длина вычисляется в том месте, где объявление ещё видно.

Индексация — это арифметика указателей

Имя массива в выражении — это адрес его первого элемента. А `a[i]` по определению языка означает `*(a + i)`.

```
int *p = a;           // то же, что &a[0]  
p[2] == *(p + 2) == a[2] == 2[a]    // всё это 30
```

`2[a]` компилируется без предупреждений, поскольку сложение коммутативно. Пользоваться такой записью не следует, но она показывает, что индексация является обычной операцией над адресом, а не свойством массива.

Индексация — это арифметика указателей

Имя массива в выражении — это адрес его первого элемента. А `a[i]` по определению языка означает `*(a + i)`.

```
int *p = a;           // то же, что &a[0]
p[2] == *(p + 2) == a[2] == 2[a]    // всё это 30
```

`2[a]` компилируется без предупреждений, поскольку сложение коммутативно. Пользоваться такой записью не следует, но она показывает, что индексация является обычной операцией над адресом, а не свойством массива.

Шаг зависит от типа

`p + 1` сдвигает адрес не на байт, а на `sizeof(*p)` байт: `int *` шагает по 4, `double *` — по 8, `char *` — по 1. Компилятор считает это за вас и подставляет нужные инструкции.

Указатель на середину — полноценный массив

```
int a[5] = {10, 20, 30, 40, 50};  
int *mid = a + 2;  
mid[0] == 30;   mid[1] == 40;   mid[-1] == 20;
```



Тот, кто получил `mid`, не может узнать ни где начало массива, ни сколько элементов осталось. Для него это просто адрес, с которого можно идти вперёд.

На этом стоит вся работа с массивами в C: функции сортировки, поиска и сдвига получают кусок массива как самостоятельный массив — адрес и длину.

Обход без индексов и один за концом

```
for (int *it = a; it != a + 5; it++)  
    printf("%d ", *it);
```

`a + 5` — адрес сразу за последним элементом. Стандарт разрешает его вычислять и сравнивать, но запрещает по нему читать: это граница, а не элемент, но на ней держится условие цикла.

Разность двух указателей внутри одного массива — расстояние в элементах:

```
int *first = &a[0], *last = &a[4];  
printf("%td\n", last - first);    // 4, хотя адреса отличаются на 16
```

Тип этой разности — `ptrdiff_t`, печатается через `%td`. Он знаковый: указатели можно вычитать в любом порядке.

Массив в аргументе функции — это адрес

```
long sum(int a[5], size_t n) {  
    printf("%zu\n", sizeof a);    // 8, а не 20  
    ...  
}
```

```
warning: 'sizeof' on array function parameter 'a' will return size of 'int *'  
[-Wsizeof-array-argument]
```

Запись `int a[5]` в списке параметров ничего не обещает: компилятор превращает её в `int *a`, и пятёрка не проверяется никогда. Массив *распадается* в указатель на первый элемент.

Главное

Поэтому все функции курса принимают пару: **указатель и длину**. Длину носите за собой вы. Никакого способа спросить её у массива нет.

Границ у массива нет

```
int a[4] = {10, 20, 30, 40};  
a[7] = -1;           // компилируется; санитайзера нет — тишина
```

Что произойдёт, зависит от того, что лежит по этому адресу: чужая переменная, выравнивание или ничего интересного. Программа может напечатать верный ответ, испортить соседнее значение или упасть — всё это на одном источнике.

Неопределённое поведение

```
ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7b8fbe3f0070  
READ of size 4 at 0x7b8fbe3f0070 thread T0  
#0 in main 11-out-of-bounds.c:35  
[96, 112) 'a' (line 22) <== Memory access at offset 112 overflows this
```

Санитайзер называет файл, строку, переменную и на сколько за неё **ВЫШЛИ**.

Санитайзер проверяет запуск, а не код

Главное о санитайзере

Санитайзер видит только то, что программа действительно сделала на этом запуске. Индекс не вышел за границу на ваших данных — он промолчит, даже если на других данных выйдет.

Из этого следуют две вещи.

Тишина санитайзера — не доказательство корректности. Она означает ровно одно: на этом входе ошибки памяти не случилось.

Проверять следует краевые случаи: пустой массив, один элемент, индекс на границе, максимальная длина. На этих входах программы ломаются чаще всего, и они же выносятся на защиту.

const: обещание не менять

```
long sum(const int *a, size_t n); // функция только читает
```

Попытка написать `a[0] = 0` внутри такой функции не даст собрать программу:

```
error: assignment of read-only location '*a'
```

Слово стоит слева от звёздочки, поэтому запрещает менять *данные*. Если поставить его справа, запрет переедет на сам указатель:

<code>const int *p</code>	менять <code>*p</code> нельзя, менять <code>p</code> можно
<code>int *const p</code>	менять <code>*p</code> можно, менять <code>p</code> нельзя

`const` ставится везде, где функция только читает: это документация, которую проверяет компилятор.

Массив нельзя присвоить

```
int a[3] = {1, 2, 3};  
int b[3];  
b = a;           // error: assignment to expression with array type
```

Копирование массива в С — это всегда цикл или вызов функции:

```
for (size_t i = 0; i < 3; i++) b[i] = a[i];
```

А вот это компилируется и делает совсем другое:

```
int *c = a;    // c показывает на тот же массив  
c[0] = 999;    // a[0] тоже стал 999
```

Одна звёздочка отделяет копию от общего доступа.

Размер массива на стеке — константа

```
int n;  
scanf("%d", &n);  
int a[n];           // warning: ISO C90 forbids variable length array [-Wvla]
```

Такие массивы в C99 есть, но на курсе запрещены флагом `-Wvla`.
Причин две: при вводе миллиарда программа завершится аварийно, и проверить это заранее негде, а кроме того, VLA позволяет обойти стороной динамическую память, которая вводится завтра.

Главное

Массив, размер которого известен только во время работы, — это завтра: `malloc`, `realloc`, `free` и свой динамический массив, который умеет расти.

РАЗДЕЛ 4

Ответ на вопрос дня

Как устроен питоновский список

```
>>> import sys
>>> [sys.getsizeof([0] * k) for k in range(5)]
[56, 64, 72, 80, 88]
```

Пустой список — 56 байт, каждый следующий элемент — ровно 8. Восемь байт независимо от того, что в нём лежит: единица, строка на мегабайт или другой список.

Как устроен питоновский список

```
>>> import sys
>>> [sys.getsizeof([0] * k) for k in range(5)]
[56, 64, 72, 80, 88]
```

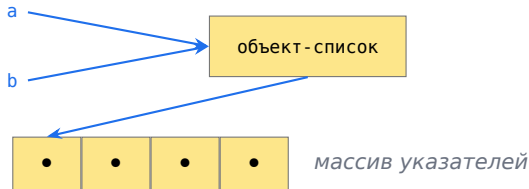
Пустой список — 56 байт, каждый следующий элемент — ровно 8. Восемь байт независимо от того, что в нём лежит: единица, строка на мегабайт или другой список.

Восемь байт — это `sizeof` указателя. Список хранит не объекты, а **адреса объектов**: внутри него лежит массив указателей, ровно такой, какой вы объявляете строкой `int *a[8]`.

Сами числа лежат отдельно, каждое — полноценный объект с заголовком. Вчера мы считали: `sys.getsizeof(0)` равен 28.

Что делает `b = a`

```
a = [1, 2, 3]
b = a
b.append(4)
```



`b = a` копирует один адрес — адрес объекта-списка. Ни массив указателей, ни сами числа при этом не копируются, поэтому `a is b` истинно и любое изменение видно с обеих сторон.

Тот же эффект даёт `int *b = a;` в C.

Копия и копия копии

```
a = [[1], [2]]
b = a[:]          # копия
b[0].append(9)
a                 # [[1, 9], [2]]
a[0] is b[0]      # True
```

```
/* копируем адреса */
for (i...) dst[i] = src[i];

/* копируем значения */
for (i...) {
    storage[i] = *src[i];
    dst[i] = &storage[i];
}
```

`a[:]` действительно создал новый массив указателей, и `b.append` больше не трогает `a`. Но указатели в нём те же самые, и вложенный список у `a` и `b` общий.

Такое копирование называется **поверхностным**: скопирован только верхний слой адресов.

Почему у массива в С нет len()

Python

- объект знает свою длину;
- знает вместимость;
- умеет расти;
- хранит адреса элементов.

`len(a)` читает готовое число.

С

- массив — это байты подряд;
- длина известна компилятору, пока видно объявление;
- в функцию доезжает только адрес.

`len` читать неоткуда.

Главное

Питоновский список — это *структура*, у которой рядом с данными лежат длина и вместимость. Завтра вы объявите такую структуру сами, положите в неё указатель и два числа — и получите `list` со всеми его свойствами, включая амортизированный `append`.

Итог дня

- `&x` — адрес переменной, `*p` — значение по адресу. Указатель — обычная переменная, в которой лежит номер байта.
- Локальные переменные живут в кадре вызова и исчезают вместе с ним. Адрес освободившегося кадра — висячий указатель, обращение по нему является UB.
- Массив — это объекты подряд, без длины и без границ. `a[i]` означает `*(a + i)`, шаг зависит от типа.
- В функцию массив попадает как адрес. Длину передаёте вы.
- Питоновский список — массив адресов, поэтому `b = a` копирует один указатель, а `a[:]` — только верхний слой.

Задачи дня: `day2-pointers-arrays/tasks/`

Вопросы к защите открыты: `defense/questions.md`, посмотрите их до того, как начнёте решать.